Memento Pattern

Question : implement an undo() method in your editor class

```
class editor {
     public:
     editor(char c) :content(c) {}
     void setcontent(char c) {
          content = c;
     }
     char getcontent()
     {
          return content;
     }
     private:
     char content;
};

int main () {
     editor editor1('a');
     editor1.setcontent('b');
     editor1.setcontent('c');
     editor1.undo(); // How can you provide this feature ?
     editor1.undo();
     return 0;
}
```

Solution I

```
class editor {
public:
     editor(char c) :content(c) { prevContent.push_back(c); }
     void setcontent(char c) {
          prevContent.push_back(c);
          content = c;
     }
     void undo()
     {
          content = prevContent.at(prevContent.size() - 1);
          prevContent.pop_back();
     }
     char getcontent() {
          return content;
     }
     private:
     char content;
     std::vector<char> prevContent;
};

int main() {
     editor editor1('a');
     editor1.setcontent('b');
     editor1.setcontent('c');
     editor1.undo();
     editor1.undo();
     return 0;
}
```

-SOLID principle violation
-editor class is violating **Single responsibility** principle
-editor should do only one thing, but here editor is also maintaining previous states

Solution II

```
class editor {
public:
	editor(char c) :content(c) {}
	void setcontent(char c) {
		content = c;
	}
	char getcontent() {
		return content;
	}
	editor getstate() {
	editor returnstate(content);
		return returnstate;
	}
	void undo(editor e)
	{
		content = e.content;
     }
private:
	char content;
};

class history {
private:
	std::vector<editor> editorhistory;
public:
	void push(editor e)
	{
		editorhistory.push_back(e);
	}
	editor pop()
	{
		editorhistory.pop_back();
		if (editorhistory.size() >= 1)
		{
		editor e = editorhistory.at(editorhistory.size() - 1);
		return e;
		}
	}
};
```

```
int main() {
editor editor1('a');
history editorh;
editorh.push(editor1.getstate());
editor1.setcontent('b');
editorh.push(editor1.getstate());
editor1.setcontent('c');
editorh.push(editor1.getstate());
editor1.undo(editorh.pop());
std::cout<<editor1.getcontent();
return 0;
}
```

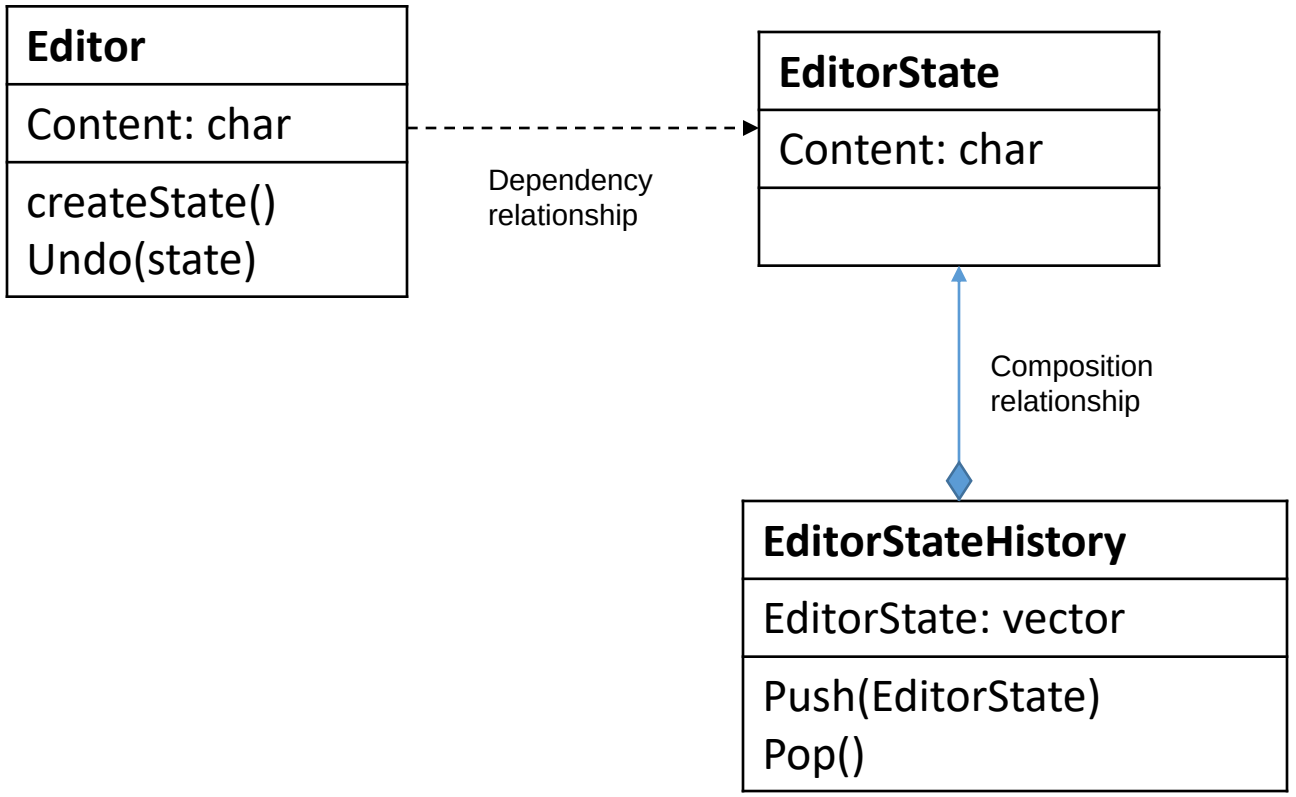
-not all state of editor class needs to be maintained.
-editor may have more member variables,
that need not be maintained in history
-current approach will make history class very bulky in real situations

Solution III

```
class editorstate {
public:
	editorstate(char c) :content(c) {}
	char getcontent()
	{
	return content;
	}
private:
		char content;
};
//editor -----> editorstate has dependency relationship
//meaning one of the editor function takes editorstate as
an argument or return value
class editor {
public:
	editor(char c) :content(c) {}
	void setcontent(char c) {
		content = c;
	}
	char getcontent() {
		return content;
	}

	editorstate getstate() {
		editorstate returnstate(this->content);
		return returnstate;
	}
	void undo(editorstate e)
	{
		content = e.getcontent();
	}
private:
	 char content;
};
```

```
//history diomond---->triangle editorstate is composition relationship
class history {
private:
	std::vector<editorstate> editorhistory;
public:
	void push(editorstate e)
	{
		editorhistory.push_back(e);
	}
	editorstate pop()
	{
		editorhistory.pop_back();
		if (editorhistory.size() >= 1)
		{
		editorstate e = editorhistory.at(editorhistory.size() - 1);
		return e;
		}
	}
};

int main() {
	editor editor1('a');
	history editorh;
	editorh.push(editor1.getstate());
	editor1.setcontent('b');
	editorh.push(editor1.getstate());
	editor1.setcontent('c');
	editorh.push(editor1.getstate());
	editor1.undo(editorh.pop());
	std::cout << editor1.getcontent();

return 0;
}
```

Memento Pattern (to implement undo() function)



Memento Pattern